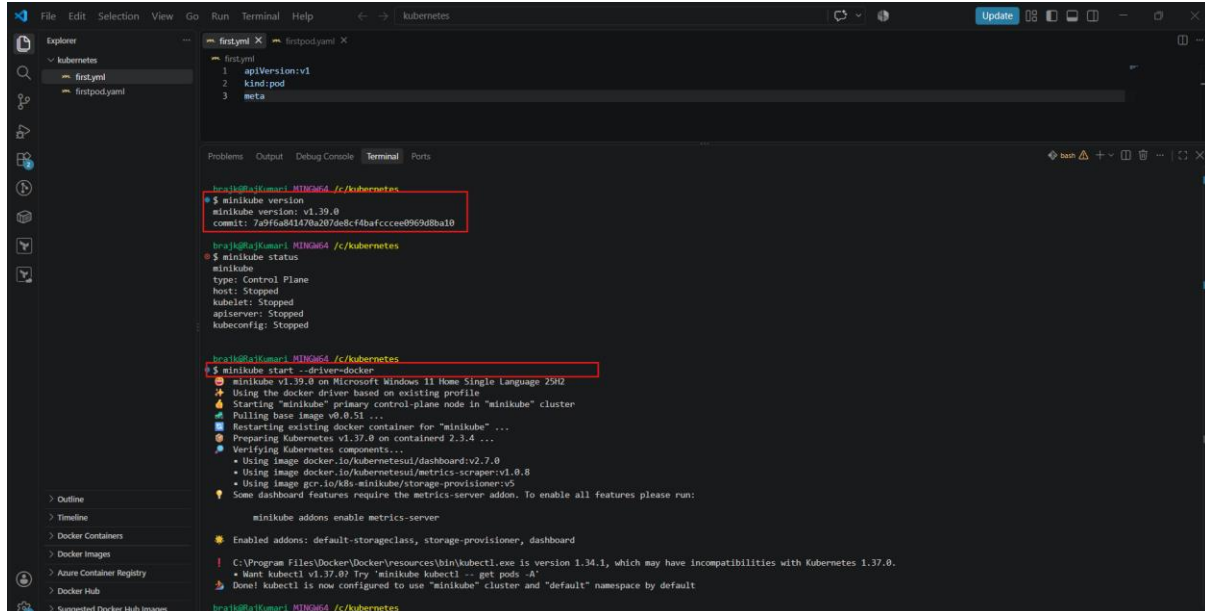


KUBERNETES (K8S) TASK - 09

1. Execute all the steps shown in video.



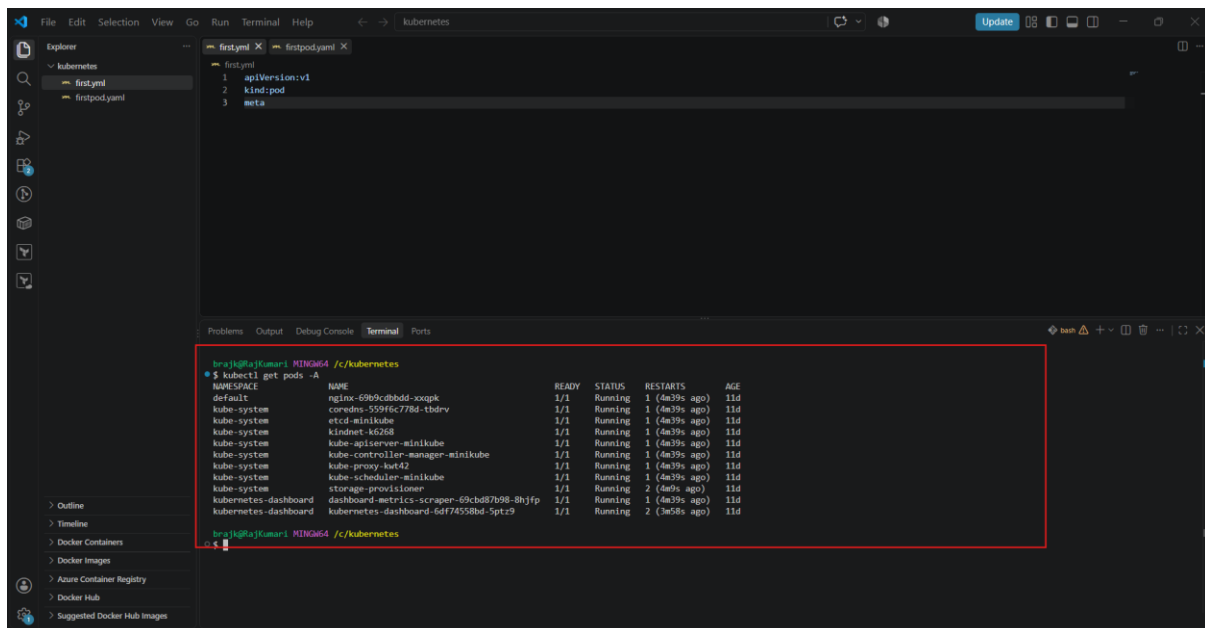
The screenshot shows a VS Code editor with a file explorer on the left containing 'first.yaml' and 'firstpod.yaml'. The terminal window is open, showing the following commands and output:

```
brajk@ajkumari-MINGW64 /c/kubernetes
$ minikube version
minikube version: v1.39.0
commit: 7a096a81470a207de8cf4bafccce0969d8ba10

brajk@ajkumari-MINGW64 /c/kubernetes
$ minikube status
minikube
type: Control Plane
host: Stopped
kubernetes: Stopped
apiserver: Stopped
kubeconfig: Stopped

brajk@ajkumari-MINGW64 /c/kubernetes
$ minikube start --driver=docker
minikube v1.39.0 on Microsoft Windows 11 Home Single Language 25H2
Using the docker driver based on existing profile
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.51 ...
Restarting existing docker container for "minikube" ...
Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
Verifying Kubernetes components...
  * Using image docker.io/kubernetes/dashboard:v2.7.0
  * Using image docker.io/kubernetes/metrics-scraper:v1.0.8
  * Using image gcr.io/k8s-minikube/storage-provisioner:v2
Some dashboard features require the metrics-server addon. To enable all features please run:
    minikube addons enable metrics-server
Enabled addons: default-storageclass, storage-provisioner, dashboard
C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.34.1, which may have incompatibilities with Kubernetes 1.37.0.
Want kubectl v1.37.0? Try 'minikube kubectl -- get pods -A'
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

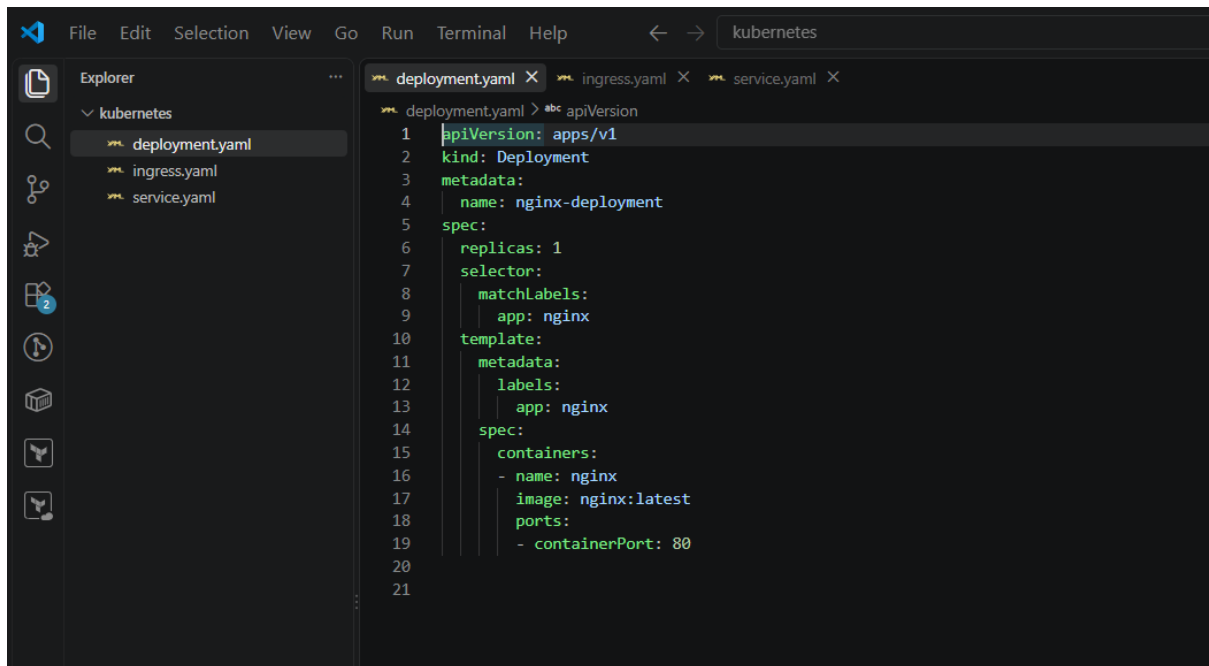
brajk@ajkumari-MINGW64 /c/kubernetes
```



The screenshot shows the same VS Code editor and terminal window. The terminal now displays the output of the command 'kubectl get pods -A', which lists all pods in the cluster:

```
brajk@ajkumari-MINGW64 /c/kubernetes
$ kubectl get pods -A
NAME                                READY   STATUS    RESTARTS   AGE
default/nginx-6989cd8dd-xouk        1/1     Running   1           11s
kube-system/coredns-559f6c778d-tbdrv  1/1     Running   1           11s
kube-system/etcd-minikube            1/1     Running   1           11s
kube-system/kindnet-k6268             1/1     Running   1           11s
kube-system/kube-apiserver-minikube  1/1     Running   1           11s
kube-system/kube-controller-manager-minikube  1/1     Running   1           11s
kube-system/kube-proxy-hwt42          1/1     Running   1           11s
kube-system/kube-scheduler-minikube  1/1     Running   1           11s
kube-system/storage-provisioner       1/1     Running   2           11s
kubernetes-dashboard/dashboard-metrics-scraper-69cbd87698-8hjfp  1/1     Running   1           11s
kubernetes-dashboard/kubernetes-dashboard-6df74558bd-5ptr9  1/1     Running   2           11s
```

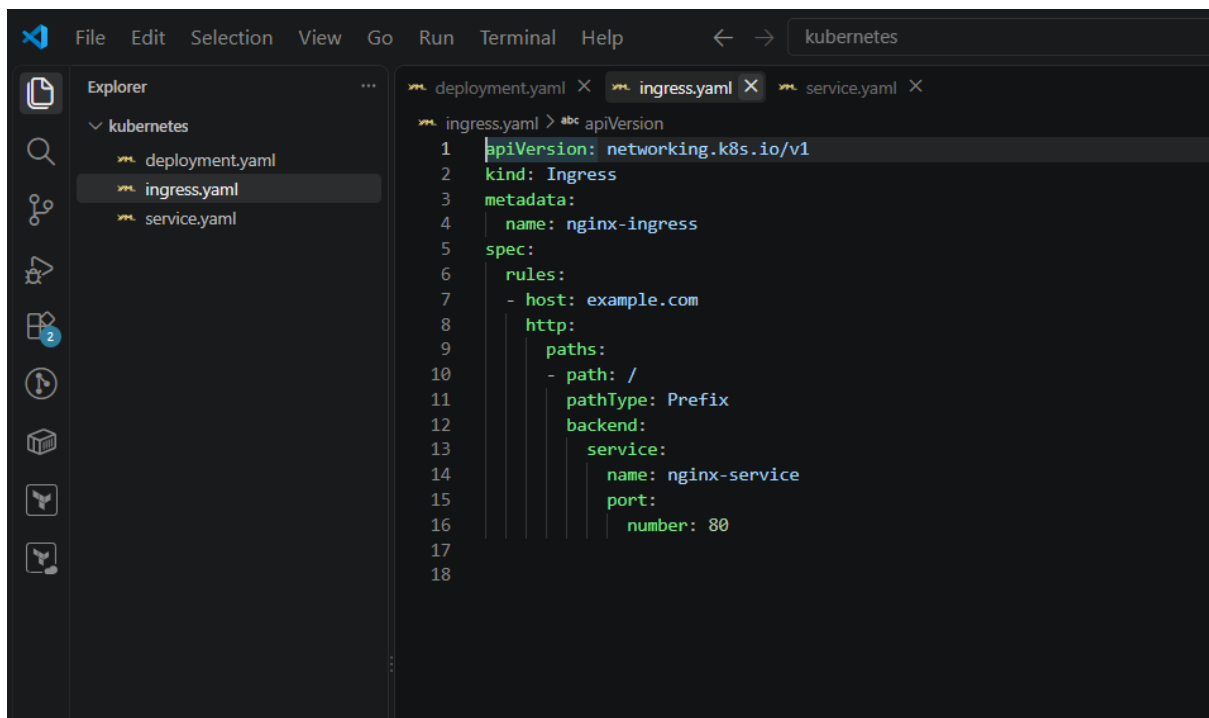
Vi deployment.yaml



The screenshot shows the Visual Studio Code editor with the 'deployment.yaml' file open. The Explorer sidebar on the left shows a project named 'kubernetes' containing three files: 'deployment.yaml', 'ingress.yaml', and 'service.yaml'. The 'deployment.yaml' file is selected and its content is displayed in the main editor. The code is a Kubernetes Deployment manifest for an nginx application. The 'apiVersion' is 'apps/v1', the 'kind' is 'Deployment', and the 'name' is 'nginx-deployment'. It specifies 1 replica and a selector with the label 'app: nginx'. The template includes a container named 'nginx' using the 'nginx:latest' image, with port 80 exposed.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx-deployment
5 spec:
6   replicas: 1
7   selector:
8     matchLabels:
9       app: nginx
10  template:
11    metadata:
12      labels:
13        app: nginx
14    spec:
15      containers:
16      - name: nginx
17        image: nginx:latest
18        ports:
19        - containerPort: 80
20
21
```

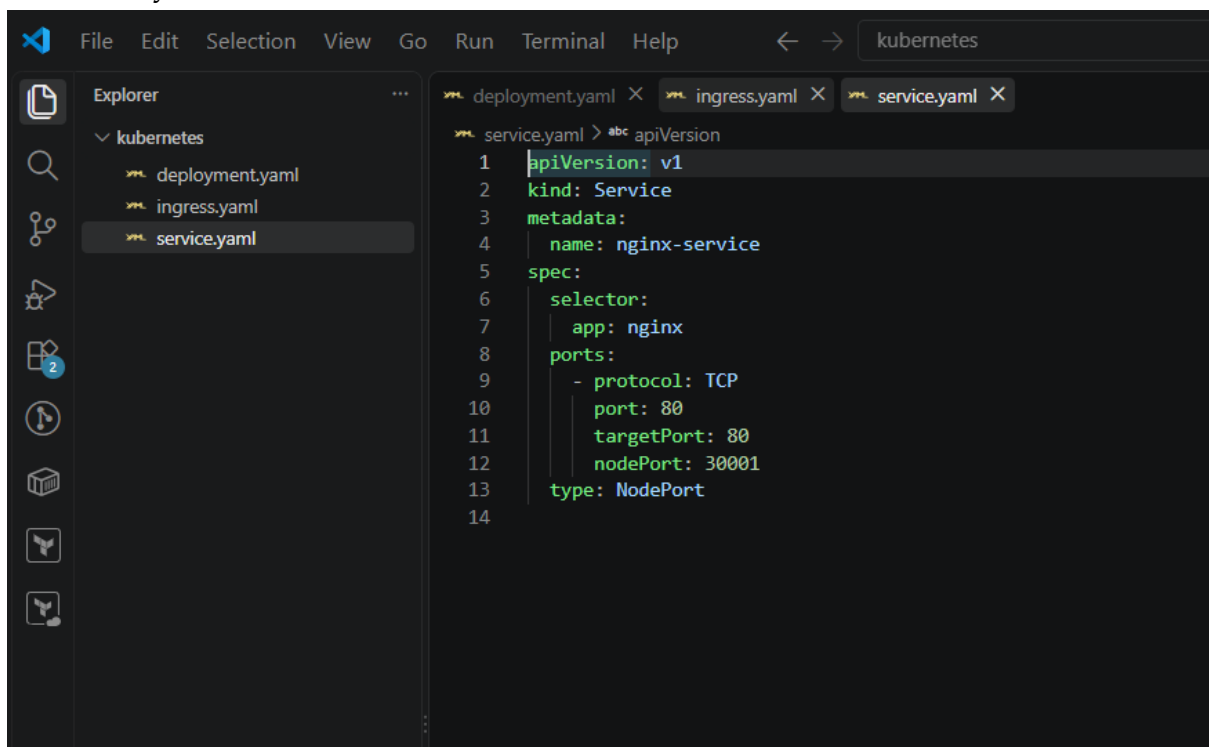
Vi ingress.yaml



The screenshot shows the Visual Studio Code editor with the 'ingress.yaml' file open. The Explorer sidebar on the left shows the same 'kubernetes' project with 'deployment.yaml', 'ingress.yaml', and 'service.yaml'. The 'ingress.yaml' file is selected and its content is displayed in the main editor. The code is a Kubernetes Ingress manifest for an nginx service. The 'apiVersion' is 'networking.k8s.io/v1', the 'kind' is 'Ingress', and the 'name' is 'nginx-ingress'. It defines a rule for the host 'example.com' with a path '/' of type 'Prefix', which routes traffic to the 'nginx-service' on port 80.

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: nginx-ingress
5 spec:
6   rules:
7   - host: example.com
8     http:
9       paths:
10      - path: /
11        pathType: Prefix
12        backend:
13          service:
14            name: nginx-service
15            port:
16              number: 80
17
18
```

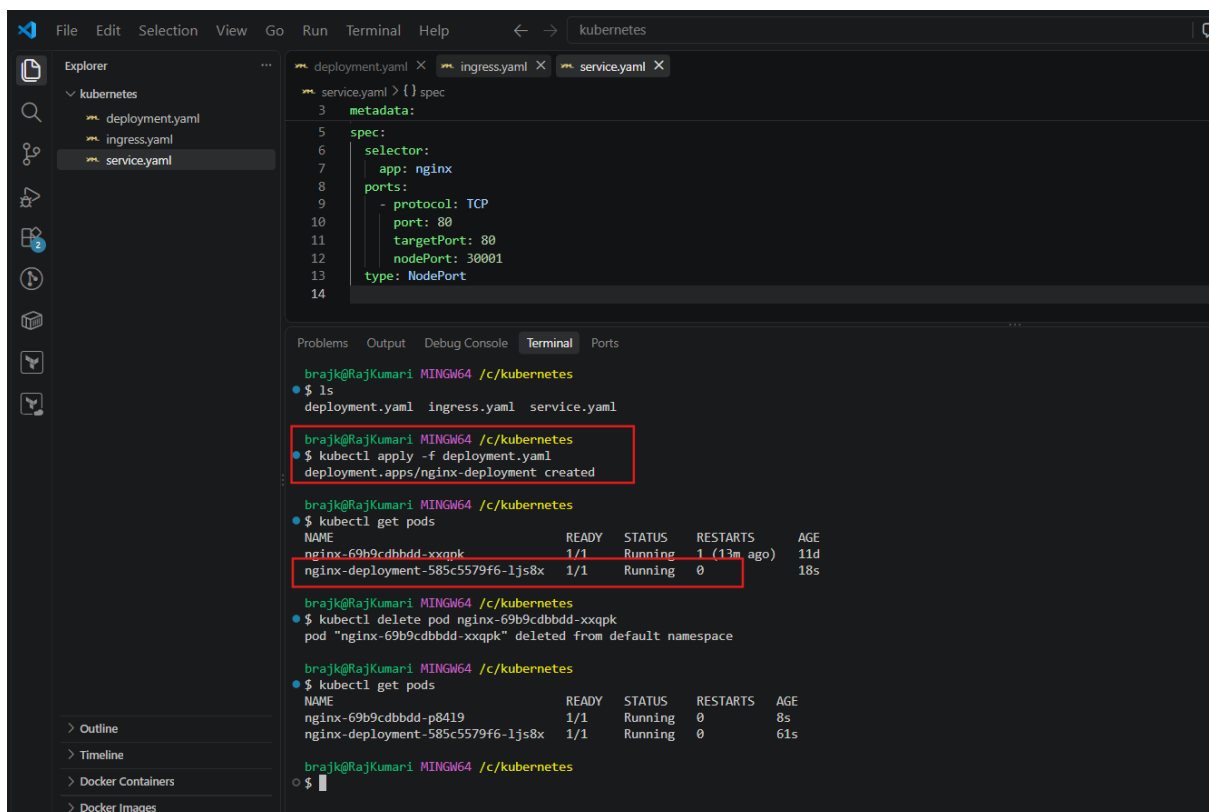
Vi service.yaml



```
File Edit Selection View Go Run Terminal Help
kubernetes

Explorer
  kubernet
    deployment.yaml
    ingress.yaml
    service.yaml

service.yaml > abc apiVersion
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: nginx-service
5  spec:
6    selector:
7      app: nginx
8    ports:
9      - protocol: TCP
10        port: 80
11        targetPort: 80
12        nodePort: 30001
13  type: NodePort
14
```



```
File Edit Selection View Go Run Terminal Help
kubernetes

Explorer
  kubernet
    deployment.yaml
    ingress.yaml
    service.yaml

service.yaml > {} spec
3  metadata:
5  spec:
6    selector:
7      app: nginx
8    ports:
9      - protocol: TCP
10        port: 80
11        targetPort: 80
12        nodePort: 30001
13  type: NodePort
14

Problems Output Debug Console Terminal Ports

brajk@RajKumari MINGW64 /c/kubernetes
$ ls
deployment.yaml ingress.yaml service.yaml

brajk@RajKumari MINGW64 /c/kubernetes
$ kubectl apply -f deployment.yaml
deployment.apps/nginx-deployment created

brajk@RajKumari MINGW64 /c/kubernetes
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-69b9cddb-xxqpk                1/1     Running   1 (13m ago)  11d
nginx-deployment-585c5579f6-ljs8x    1/1     Running   0           18s

brajk@RajKumari MINGW64 /c/kubernetes
$ kubectl delete pod nginx-69b9cddb-xxqpk
pod "nginx-69b9cddb-xxqpk" deleted from default namespace

brajk@RajKumari MINGW64 /c/kubernetes
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-69b9cddb-p8419                1/1     Running   0           8s
nginx-deployment-585c5579f6-ljs8x    1/1     Running   0           61s

brajk@RajKumari MINGW64 /c/kubernetes
$
```

The screenshot shows a VS Code editor with a file explorer on the left containing 'deployment.yaml', 'ingress.yaml', and 'service.yaml'. The main editor displays the 'service.yaml' file with the following content:

```
3 metadata:
5 spec:
6   selector:
7     app: nginx
8   ports:
9     - protocol: TCP
10       port: 80
11       targetPort: 80
12       nodePort: 30001
13   type: NodePort
14
```

Below the editor, the 'Terminal' tab shows the following commands and output:

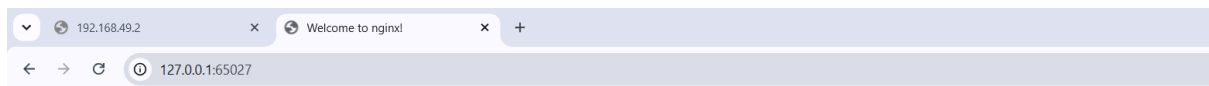
```
brajk@rajKumari MINGW64 /c/kubernetes
$ cat service.yaml
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30001
  type: NodePort

brajk@rajKumari MINGW64 /c/kubernetes
$ kubectl apply -f service.yaml
service/nginx-service created

brajk@rajKumari MINGW64 /c/kubernetes
$ kubectl get svc
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes    ClusterIP   10.96.0.1    <none>         443/TCP          11d
nginx         NodePort    10.101.91.165 <none>         80:32497/TCP     11d
nginx-service NodePort    10.109.44.56 <none>         80:30001/TCP     8s

brajk@rajKumari MINGW64 /c/kubernetes
$ minikube ip
192.168.49.2

brajk@rajKumari MINGW64 /c/kubernetes
$ minikube service nginx-service --url
http://127.0.0.1:65027
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to fs.com/nginx.

Thank you for using nginx.

The image shows a VS Code editor with three files open: `deployment.yaml`, `ingress.yaml`, and `service.yaml`. The `ingress.yaml` file is selected and shows the following content:

```
1 deployment.yaml X ingress.yaml X service.yaml X
2
3 metadata:
4
5 spec:
6 selector:
7   app: nginx
8 ports:
9   - protocol: TCP
10     port: 80
11     targetPort: 80
12     nodePort: 30001
13 type: NodePort
14
```

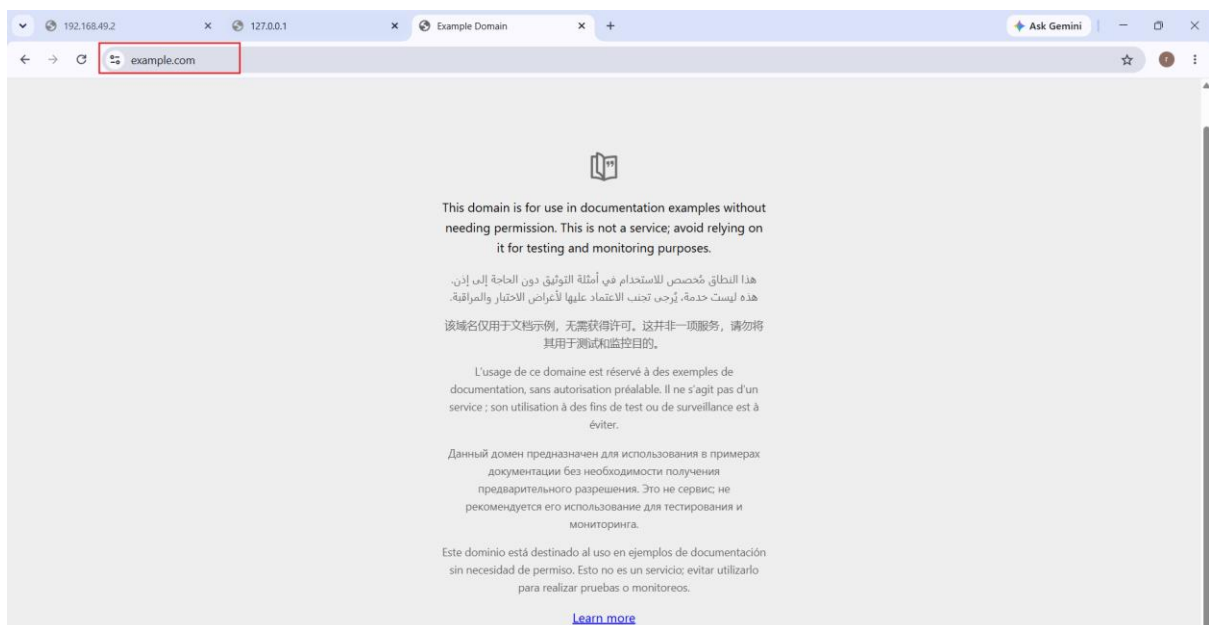
The terminal window shows the following commands and output:

```
brajk@RajKumar1: ~/kubernetes
$ kubectl apply -f ingress.yaml

brajk@RajKumar1: ~/kubernetes
$ minikube addons enable ingress
✖ ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
! After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
• Using image registry.k8s.io/nginx-kube-webhook-certgen:v1.6.9
• Using image registry.k8s.io/nginx-kube-webhook-certgen:v1.6.9
• Using image registry.k8s.io/nginx-kube-webhook-certgen:v1.6.9
• Verifying ingress addon...
✔ The "ingress" addon is enabled

brajk@RajKumar1: ~/kubernetes
$ minikube addons enable ingress
✖ ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
! After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
• Using image registry.k8s.io/nginx-kube-webhook-certgen:v1.6.9
• Using image registry.k8s.io/nginx-kube-webhook-certgen:v1.6.9
• Using image registry.k8s.io/nginx-kube-webhook-certgen:v1.6.9
• Verifying ingress addon...
✔ The "ingress" addon is enabled

brajk@RajKumar1: ~/kubernetes
$ kubectl get ingress
NAME          CLASS    HOSTS          ADDRESS          PORTS    AGE
nginx-ingress  <none>   example.com    192.168.49.2     80       14m
```



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

2. Upgrade k8s cluster from 1.26 to 1.28 Version. Note: Document to upgrade cluster.

<https://devopscube.com/upgrade-kubernetes-cluster-kubeadm/>

Pre-Upgrade Checks

Step 1: Check cluster version

From the control-plane node:

kubectl version

Better:

kubectl get nodes -o wide

kubeadm version

kubelet --version

kubectl version --client

```
aws [Alt+S] [Search] [Alt+S] United States (N. Virginia) [backspace]

abuntus@k8s-master:~$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"31", BuildDate:"2024-08-20T15:17:54Z", GoVersion:"go1.26.5", Compiler:"gc", Platform:"linux/amd64"}
abuntus@k8s-master:~$ kubelet --version
Kubernetes v1.34.11
abuntus@k8s-master:~$ kubectl version --client
Client Version: v1.34.11
abuntus@k8s-master:~$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
k8s-master          Ready     control-plane  12d   v1.34.11
k8s-worker1         Ready     <none>      12d   v1.34.11
k8s-worker2         Ready     <none>      12d   v1.34.11
abuntus@k8s-master:~$ kubectl get pods -A
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE
default     probe-pod                             1/1     Running   2 (20m ago)  3d2h
kube-system calico-kube-controllers-f4cf54b75-bhgjr 0/1     CrashLoopBackOff  358 (2m54s ago)  12d
kube-system calico-node-7h9pp             1/1     Running    10 (20m ago)  12d
kube-system calico-node-bpw4p             1/1     Running    11 (20m ago)  12d
kube-system calico-node-hgltg             1/1     Running    10 (20m ago)  12d
kube-system coredns-64bc5c9577-s4zsp      1/1     Running    10 (20m ago)  12d
kube-system coredns-64bc5c9577-admpt      1/1     Running    10 (20m ago)  12d
kube-system ebs-csi-controller-5c47f7d4d6-h6pak 6/6     Running    6 (20m ago)  3d2h
kube-system ebs-csi-controller-5c47f7d4d6-q4bqj 6/6     Running    6 (20m ago)  3d3h
kube-system ebs-csi-node-5g52g            3/3     Running    9 (20m ago)  3d18h
kube-system ebs-csi-node-ff9gr            3/3     Running    6 (20m ago)  3d18h
kube-system ebs-csi-node-l6dhs            3/3     Running    7 (20m ago)  3d18h
kube-system ebs-csi-controller-5c774b7686-7a7dk 3/3     Running    3 (20m ago)  3d3h
kube-system ebs-csi-controller-5c774b7686-kdc45 3/3     Running    3 (20m ago)  3d3h
kube-system ebs-csi-node-jfow             3/3     Running    6 (20m ago)  3d15h
kube-system ebs-csi-node-pmzrv            3/3     Running    6 (20m ago)  3d15h
kube-system ebs-csi-node-zkfgx            3/3     Running    6 (20m ago)  3d15h
kube-system etcd-k8s-master               1/1     Running    10 (20m ago)  12d
kube-system kube-apiserver-k8s-master     1/1     Running    10 (20m ago)  12d
kube-system kube-controller-manager-k8s-master 1/1     Running    10 (20m ago)  12d
kube-system kube-proxy-nmcc              1/1     Running    10 (20m ago)  12d
kube-system kube-proxy-h8fls             1/1     Running    11 (20m ago)  12d
kube-system kube-proxy-wqgvs             1/1     Running    10 (20m ago)  12d
kube-system kube-scheduler-k8s-master     1/1     Running    10 (20m ago)  12d
kube-system metrics-server-7c5fd4664-8vdh2 1/1     Running    1 (20m ago)  3d2h
abuntus@k8s-master:~$ kubectl get pods -n kube-system
```

```
ubuntu@k8s-master:~$ kubeadm version -o json
{
  "clientVersion": {
    "major": "1",
    "minor": "34",
    "gitVersion": "v1.34.11",
    "gitCommit": "3a634765b787dd069f7f714fa77d767cb7d43795",
    "gitTreeState": "clean",
    "buildDate": "2026-08-20T15:17:54Z",
    "goVersion": "go1.26.5",
    "compiler": "gc",
    "platform": "linux/amd64"
  }
}
ubuntu@k8s-master:~$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.11", GitCommit:"3a634765b787dd069f7f714fa77d767cb7d43795", TreeState:"clean", BuildDate:"2026-08-20T15:17:54Z", GoVersion:"go1.26.5", Compiler:"gc", Platform:"linux/a
}
ubuntu@k8s-master:~$
```

Step 5.1 — Upgrade kubeadm on Control Plane

Login to:

```
ssh ubuntu@k8s-master
```

Check:

```
kubeadm version
```

Unhold:

```
sudo apt-mark unhold kubeadm
```

Update package information:

```
sudo apt-get update
```

Check available kubeadm versions:

```
sudo apt-cache madison kubeadm
```

You should look for a **1.27.x** release.

For example:

```
kubeadm | 1.27.x-...
```

Install the desired 1.27 patch version:

```
sudo apt-get install -y kubeadm='1.27.x-*'
```

Then hold it:

sudo apt-mark hold kubeadm

Verify:

kubeadm version

aws

Search

[Alt+S]

United States (N. Virginia)

badugu (820982384063)

badugu

4

[upgrade/versions] Target version: v1.34.12

[upgrade/versions] Latest version in the v1.34 series: v1.34.12

Components that must be upgraded manually after you have upgraded the control plane with 'kubeadm upgrade apply':

COMPONENT	NODE	CURRENT	TARGET
kubelet	k8s-master	v1.34.11	v1.34.12
kubelet	k8s-worker1	v1.34.11	v1.34.12
kubelet	k8s-worker2	v1.34.11	v1.34.12

Upgrade to the latest version in the v1.34 series:

COMPONENT	NODE	CURRENT	TARGET
kube-apiserver	k8s-master	v1.34.11	v1.34.12
kube-controller-manager	k8s-master	v1.34.11	v1.34.12
kube-scheduler	k8s-master	v1.34.11	v1.34.12
kube-proxy		v1.34.11	v1.34.12
CoreDNS		v1.12.1	v1.12.1
etcd	k8s-master	3.6.5-0	3.6.5-0

You can now apply the upgrade by executing the following command:

kubeadm upgrade apply v1.34.12

The table below shows the current state of component configs as understood by this version of kubeadm. Configs that have a "yes" mark in the "MANUAL UPGRADE REQUIRED" column require manual config upgrade or resetting to kubeadm defaults before a successful upgrade can be performed. The version to manually upgrade to is denoted in the "PREFERRED VERSION" column.

API GROUP	CURRENT VERSION	PREFERRED VERSION	MANUAL UPGRADE REQUIRED
kubeproxy.config.k8s.io	v1alpha1	v1alpha1	no
kubelet.config.k8s.io	v1beta1	v1beta1	no

ubuntu@k8s-master:~\$

i-01b64330d55a2dc07 (k8s-master)

PublicIPs: 98.93.255.188 PrivateIPs: 172.31.17.127

```
aws | [Icons] | Search [Alt+S] | [Icons] | [Icons] | [Icons] | [Icons] | United States (N. Virginia) | badugu (820982384063) | badugu | S

ubuntu@k8s-master:~$ kubelet --version
kubelet version
Kubernetes v1.34.12
Client Version: v1.34.12
Kustomize Version: v5.7.1
ubuntu@k8s-master:~$ kubectl uncordon k8s-master
node/k8s-master uncordoned
ubuntu@k8s-master:~$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-master    Ready     control-plane  12d   v1.34.12
k8s-worker1   Ready     <none>      12d   v1.34.11
k8s-worker2   Ready     <none>      12d   v1.34.11
ubuntu@k8s-master:~$
```

Kubernetes 1.34.11 → 1.34.12 using kubeadm

1. Upgrade architecture

Assume this cluster:

Node	Role	Current	Target
k8s-master	Control Plane	1.26	1.28
k8s-worker1	Worker	1.26	1.28
k8s-worker2	Worker	1.26	1.28

Upgrade sequence:

Kubernetes 1.26

|

v

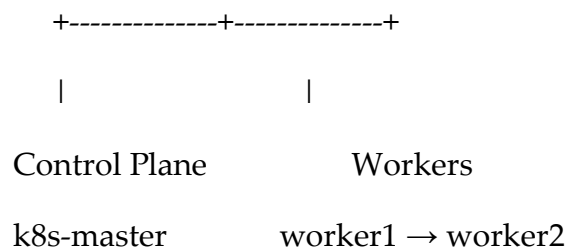
Upgrade to 1.27

|

v

Upgrade to 1.28

|



The recommended high-level workflow is to upgrade the primary control plane first and then worker nodes.

PART 1 — Pre-Upgrade Checks

Step 1: Check cluster version

From the control-plane node:

```
kubectl version
```

Better:

```
kubectl get nodes -o wide
```

Expected:

NAME	STATUS	ROLES	VERSION
k8s-master	Ready	control-plane	v1.26.x
k8s-worker1	Ready	<none>	v1.26.x
k8s-worker2	Ready	<none>	v1.26.x

Also check:

```
kubeadm version
```

```
kubelet --version
```

```
kubectl version --client
```

PART 2 — Check Cluster Health

Before upgrading, make sure everything is healthy.

```
kubectl get nodes
```

All nodes should be:

Ready

Check pods:

```
kubectl get pods -A
```

Check system pods:

```
kubectl get pods -n kube-system
```

Check cluster components:

```
kubectl get --raw='/readyz?verbose'
```

You want the cluster to be healthy **before** starting the upgrade.

Also check:

```
kubectl get events -A --sort-by='.lastTimestamp'
```

Important

Do not start an upgrade if you already have major problems such as:

NotReady

CrashLoopBackOff

Pending

Failed

DiskPressure

MemoryPressure

NetworkUnavailable

This is especially important for your EC2 lab because you've previously encountered **DiskPressure**. Clean up disk space before attempting an upgrade.

PART 3 — Backup etcd

Before a production upgrade, take an etcd backup.

First find the etcd pod:

```
kubectl get pods -n kube-system | grep etcd
```

Check where etcd is running:

```
kubectl describe pod -n kube-system etcd-k8s-master
```

A typical kubeadm etcd backup command is:

```
sudo ETCDCTL_API=3 etcdctl \  
--endpoints=https://127.0.0.1:2379 \  
--cacert=/etc/kubernetes/pki/etcd/ca.crt \  
--cert=/etc/kubernetes/pki/etcd/server.crt \  
--key=/etc/kubernetes/pki/etcd/server.key \  
snapshot save /opt/etcd-backup.db
```

Verify:

```
sudo ETCDCTL_API=3 etcdctl \  
--write-out=table \  
snapshot status /opt/etcd-backup.db
```

Why backup?

If something goes wrong during the upgrade, etcd contains the Kubernetes cluster state.

PART 4 — Important Version Rule

You are starting:

1.26

Target:

1.28

Therefore:

Upgrade #1

1.26 → 1.27

Complete the entire cluster upgrade.

Then:

Upgrade #2

1.27 → 1.28

Complete the entire cluster upgrade.

Do **not** do:

1.26 → 1.28

Kubernetes explicitly does not support skipping minor versions.

PART 5 — Upgrade 1.26 → 1.27

Step 5.1 — Upgrade kubeadm on Control Plane

Login to:

```
ssh ubuntu@k8s-master
```

Check:

```
kubeadm version
```

Unhold:

```
sudo apt-mark unhold kubeadm
```

Update package information:

```
sudo apt-get update
```

Check available kubeadm versions:

```
sudo apt-cache madison kubeadm
```

You should look for a **1.27.x** release.

For example:

```
kubeadm | 1.27.x-...
```

Install the desired 1.27 patch version:

```
sudo apt-get install -y kubeadm='1.27.x-*
```

Then hold it:

```
sudo apt-mark hold kubeadm
```

Verify:

```
kubeadm version
```

PART 6 — Check Upgrade Plan

Run:

```
sudo kubeadm upgrade plan
```

This checks whether the cluster is upgradeable and shows available versions.

You should see something similar to:

Components that must be upgraded manually after you have upgraded the control plane with 'kubeadm upgrade apply':

COMPONENT	CURRENT	TARGET
kubelet	1.26.x	1.27.x

PART 7 — Upgrade Control Plane

Now run:

```
sudo kubeadm upgrade apply v1.27.x
```

Replace x with the exact patch version you selected.

For example, if your repository provides:

1.27.16

then:

```
sudo kubeadm upgrade apply v1.27.16
```

Wait for:

[upgrade/successful] SUCCESS!

At this point:

kubeadm

↓

Control Plane components

have been upgraded.

But **kubelet is not yet upgraded.**

PART 8 — Drain Control Plane

From a terminal with kubectl access:

```
kubectl drain k8s-master --ignore-daemonsets
```

If you have pods using emptyDir, you may encounter:

cannot delete Pods with local storage

Then, after confirming that deleting that temporary data is acceptable:

```
kubectl drain k8s-master \
```

```
--ignore-daemonsets \
```

```
--delete-emptydir-data
```

Draining marks the node unschedulable and evicts workloads so the node can be safely maintained.

PART 9 — Upgrade kubelet and kubectl

On k8s-master:

```
sudo apt-mark unhold kubelet kubectl
```

Then:

```
sudo apt-get update
```

Install the 1.27 versions:

```
sudo apt-get install -y \
```

```
kubelet='1.27.x-*' \
```

```
kubectl='1.27.x-*'
```

Hold them:

```
sudo apt-mark hold kubelet kubectl
```

Restart kubelet:

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart kubelet
```

Check:

```
sudo systemctl status kubelet
```

And:

```
kubelet --version
```

```
kubectl version --client
```

The Kubernetes documentation recommends upgrading kubelet and kubectl after the control plane upgrade and restarting kubelet.

PART 10 — Uncordon Control Plane

Run:

```
kubectl uncordon k8s-master
```

Check:

```
kubectl get nodes
```

You should see:

```
k8s-master  Ready  control-plane  v1.27.x
```

PART 11 — Upgrade Worker 1

Now move to:

```
k8s-worker1
```

First drain it from the control plane:

```
kubectldrain k8s-worker1 --ignore-daemonsets
```

If required:

```
kubectldrain k8s-worker1 \
  --ignore-daemonsets \
  --delete-emptydir-data
```

On worker1 — Upgrade kubeadm

SSH:

```
ssh ubuntu@k8s-worker1
```

Check:

```
kubeadm version
```

Unhold:

```
sudo apt-mark unhold kubeadm
```

Install 1.27:

```
sudo apt-get update
```

```
sudo apt-get install -y kubeadm='1.27.x-*
```

Hold:

```
sudo apt-mark hold kubeadm
```

Upgrade worker kubeadm configuration

Run:

```
sudo kubeadm upgrade node
```

For worker nodes, kubeadm upgrade node updates the kubelet configuration rather than upgrading the control-plane components.

Upgrade worker kubelet + kubectl

```
sudo apt-mark unhold kubelet kubectl
```

```
sudo apt-get update
```

```
sudo apt-get install -y \
```

```
kubelet='1.27.x-*' \
```

```
kubectl='1.27.x-*'
```

Hold:

```
sudo apt-mark hold kubelet kubectl
```

Restart:

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart kubelet
```

Check:

```
sudo systemctl status kubelet
```

Uncordon worker1

From the control plane:

```
kubectl uncordon k8s-worker1
```

Verify:

```
kubectl get nodes
```

PART 12 — Upgrade Worker 2

Do exactly the same process.

Drain:

```
kubectl drain k8s-worker2 --ignore-daemonsets
```

If needed:

```
kubectl drain k8s-worker2 \
```

```
--ignore-daemonsets \
```

```
--delete-emptydir-data
```

SSH:

```
ssh ubuntu@k8s-worker2
```

Upgrade kubeadm:

```
sudo apt-mark unhold kubeadm
```

```
sudo apt-get update
```

```
sudo apt-get install -y kubeadm='1.27.x-*'
```

```
sudo apt-mark hold kubeadm
```

Then:

```
sudo kubeadm upgrade node
```

Upgrade kubelet/kubectl:

```
sudo apt-mark unhold kubelet kubectl
```

```
sudo apt-get update
```

```
sudo apt-get install -y \
```

```
kubelet='1.27.x-*' \
```

```
kubectl='1.27.x-*'
```

```
sudo apt-mark hold kubelet kubectl
```

Restart:

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart kubelet
```

Then:

```
kubectl uncordon k8s-worker2
```

PART 13 — Verify 1.27

Now:

```
kubectl get nodes
```

Expected:

NAME	STATUS	ROLES	VERSION
k8s-master	Ready	control-plane	v1.27.x
k8s-worker1	Ready	<none>	v1.27.x
k8s-worker2	Ready	<none>	v1.27.x

Check:

```
kubectl get pods -A
```

And:

```
kubectl get nodes -o wide
```

At this point **1.26 → 1.27 is complete.**

PART 14 — Now Upgrade 1.27 → 1.28

Now repeat the same procedure.

The sequence becomes:

1.27

|

+--> Control Plane

|

+--> Worker 1

|

+--> Worker 2

|

v

1.28

PART 15 — Control Plane 1.27 → 1.28

On k8s-master:

```
sudo apt-mark unhold kubeadm
```

```
sudo apt-get update
```

Check:

```
sudo apt-cache madison kubeadm
```

Install 1.28:

```
sudo apt-get install -y kubeadm='1.28.x-*'
```

Hold:

```
sudo apt-mark hold kubeadm
```

Check:

```
kubeadm version
```

Then:

```
sudo kubeadm upgrade plan
```

Apply:

```
sudo kubeadm upgrade apply v1.28.x
```

PART 16 — Drain Control Plane

```
kubectrl drain k8s-master --ignore-daemonsets
```

If required:

```
kubectl drain k8s-master \
  --ignore-daemonsets \
  --delete-emptydir-data
```

PART 17 — Upgrade kubelet/kubectl to 1.28

```
sudo apt-mark unhold kubelet kubectl
sudo apt-get update
sudo apt-get install -y \
  kubelet='1.28.x-*' \
  kubectl='1.28.x-*
```

Hold:

```
sudo apt-mark hold kubelet kubectl
```

Restart:

```
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

Check:

```
kubelet --version
kubectl version --client
```

Uncordon:

```
kubectl uncordon k8s-master
```

PART 18 — Worker1 1.27 → 1.28

Drain:

```
kubectl drain k8s-worker1 --ignore-daemonsets
```

SSH:

```
ssh ubuntu@k8s-worker1
```

Upgrade kubeadm:

```
sudo apt-mark unhold kubeadm
```

```
sudo apt-get update
```

```
sudo apt-get install -y kubeadm='1.28.x-*'
```

```
sudo apt-mark hold kubeadm
```

Run:

```
sudo kubeadm upgrade node
```

Upgrade kubelet and kubectl:

```
sudo apt-mark unhold kubelet kubectl
```

```
sudo apt-get update
```

```
sudo apt-get install -y \
```

```
kubelet='1.28.x-*' \
```

```
kubectl='1.28.x-*'
```

```
sudo apt-mark hold kubelet kubectl
```

Restart:

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart kubelet
```

Uncordon:

```
kubectl uncordon k8s-worker1
```

PART 19 — Worker2 1.27 → 1.28

Drain:

```
kubectldrain k8s-worker2 --ignore-daemonsets
```

SSH:

```
ssh ubuntu@k8s-worker2
```

Upgrade:

```
sudo apt-mark unhold kubeadm
```

```
sudo apt-get update
```

```
sudo apt-get install -y kubeadm='1.28.x-*
```

```
sudo apt-mark hold kubeadm
```

Then:

```
sudo kubeadm upgrade node
```

Upgrade:

```
sudo apt-mark unhold kubelet kubectld
```

```
sudo apt-get update
```

```
sudo apt-get install -y \
```

```
kubelet='1.28.x-*' \
```

```
kubectld='1.28.x-*
```

```
sudo apt-mark hold kubelet kubectld
```

Restart:

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart kubelet
```

Uncordon:

```
kubectl uncordon k8s-worker2
```

PART 20 — Final Verification

Run:

```
kubectl get nodes
```

Expected:

NAME	STATUS	ROLES	VERSION
k8s-master	Ready	control-plane	v1.28.x
k8s-worker1	Ready	<none>	v1.28.x
k8s-worker2	Ready	<none>	v1.28.x

Then:

```
kubectl get pods -A
```

Check:

```
kubectl get pods -n kube-system
```

Check services:

```
kubectl get svc -A
```

Check deployments:

```
kubectl get deployments -A
```

Check DaemonSets:

```
kubectl get daemonsets -A
```

Important: Check Your CNI

Your cluster uses **Calico**, so after the upgrade you should verify Calico.

```
kubectl get pods -n kube-system | grep calico
```

For example:

```
calico-node-xxxxx 1/1 Running
```

```
calico-node-yyyyy 1/1 Running
```

```
calico-node-zzzzz 1/1 Running
```

Also:

```
kubectl get daemonset -n kube-system
```

The Kubernetes documentation specifically notes that the CNI provider may have its own upgrade requirements and should be checked separately.

Complete Upgrade Flow

Keep this diagram for your notes:

CURRENT CLUSTER

Kubernetes

1.26

|

|

+-----v-----+

| Control Plane |

| 1.26→1.27 |

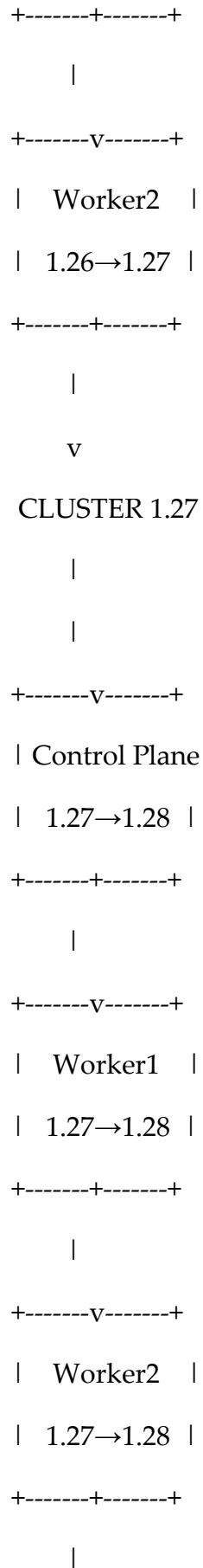
+-----+-----+

|

+-----v-----+

| Worker1 |

| 1.26→1.27 |



CLUSTER 1.28

What happens to workloads?

When you run:

```
kubectl drain k8s-worker1
```

Kubernetes attempts to move eligible workloads to other nodes.

For example:

Before drain

worker1

└─ nginx

└─ application

└─ frontend

↓ drain

worker2

└─ nginx

└─ application

└─ frontend

After the upgrade:

```
kubectl uncordon k8s-worker1
```

worker1 becomes available for scheduling again.

The official Kubernetes workflow also emphasizes draining nodes before minor-version kubelet upgrades.

Very Important for Your AWS EC2 Lab

Before you perform this exercise, check:

`df -h`

on **every node**.

Also:

`free -h`

and:

`kubectl describe node k8s-worker1 | grep -i -A5 pressure`

`kubectl describe node k8s-worker2 | grep -i -A5 pressure`

Because your previous cluster encountered:

`DiskPressure=True`

and:

node was low on resource: ephemeral-storage

I would **not start the upgrade until DiskPressure is cleared**.

Also verify:

`kubectl get nodes`

must show:

Ready

for all nodes.

aws

Search

[Alt+S]

United States (N. Virginia)

badugu (820982384063)

badugu

ubuntu@k8s-master:~\$ kubectl uncordon k8s-worker1
node/k8s-worker1 already uncordoned
ubuntu@k8s-master:~\$ kubectl get nodes
NAME STATUS ROLES AGE VERSION
k8s-master Ready control-plane 12d v1.34.12
k8s-worker1 Ready <none> 12d v1.34.12
k8s-worker2 Ready <none> 12d v1.34.11
ubuntu@k8s-master:~\$ kubectl get nodes
NAME STATUS ROLES AGE VERSION
k8s-master Ready control-plane 12d v1.34.12
k8s-worker1 Ready <none> 12d v1.34.12
k8s-worker2 Ready <none> 12d v1.34.12
ubuntu@k8s-master:~\$ kubectl get nodes -o wide
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP OS-IMAGE KERN
L-VERSION CONTAINER-RUNTIME
k8s-master Ready control-plane 12d v1.34.12 172.31.17.127 <none> Ubuntu 26.04.1 LTS 7.0.0
-1012-aws containerd://2.2.2
k8s-worker1 Ready <none> 12d v1.34.12 172.31.16.155 <none> Ubuntu 26.04 LTS 7.0.0
-1013-aws containerd://2.2.2
k8s-worker2 Ready <none> 12d v1.34.12 172.31.26.51 <none> Ubuntu 26.04 LTS 7.0.0
-1012-aws containerd://2.2.2
ubuntu@k8s-master:~\$